TASK 13 — NumPy Random Sampling

1. Task Overview

This task focuses on **random data generation and random sampling using NumPy**. It demonstrates how to generate random integers, floating-point numbers, samples from an array, and simulated student data. It also explains how a random seed makes results reproducible.

2. Objective

Understand random data generation and its importance in data science experiments. The main goal is to learn how random values can be generated, sampled, controlled with a seed, and used in practical data-science experiments.

3. Tools Used

- **Python** — programming language used to implement the examples.
- **NumPy** — numerical computing library used for random generation and sampling.
- **Jupyter Notebook** — interactive environment for executing and documenting the work.

4. What Is Random Data?

Random data consists of values generated according to a defined random process. In data science, random data can be useful for simulation, testing, sampling, experimentation, and creating synthetic datasets.

Important: NumPy random functions generate pseudo-random values. They are suitable for many data-science and simulation tasks, but they are not the same as cryptographically secure randomness.

5. Generating Random Integers

Function: `np.random.randint()`

This function generates random integer values within a specified range. The upper limit is excluded.

```
np.random.seed(42)
random_integers = np.random.randint(1, 101, size=10)
print(random_integers)
```

Explanation: 1 is the lowest possible value, 101 is the exclusive upper boundary, and `size=10` requests ten values.

6. Generating Random Floating-Point Numbers

Function: `np.random.random()`

It generates floating-point values in the interval from 0 up to, but not including, 1.

```
random_floats = np.random.random(10)
print(random_floats)
```

For a custom range, `np.random.uniform()` can be used.

```
random_values = np.random.uniform(10, 50, size=5)
print(random_values)
```

7. Random Sampling from an Array

Function: `np.random.choice()`

This function selects random values from an existing array. It is useful when a dataset contains a population and we need a random sample.

```
students = np.array(['Aman','Priya','Rahul','Neha','Karan','Sneha','Vikas','Anjali'])
sample = np.random.choice(students, size=4, replace=False)
print(sample)
```

replace=False means that the same student cannot be selected more than once in this sample.

8. Simulating a Student Marks Dataset

Random values can be used to create a small synthetic dataset when a real dataset is not available. This is useful for learning, testing, and demonstrating data-processing workflows.

```
np.random.seed(100)
marks = np.random.randint(40, 101, size=10)
print(marks)
```

Here, the generated marks are integers from 40 through 100. The fixed seed makes the generated sequence repeatable.

9. What Is a Random Seed?

A **random seed** initializes the pseudo-random number generator. When the same seed is used with the same sequence of random operations, the same results can be reproduced.

```
np.random.seed(123)
result_1 = np.random.randint(1, 50, size=5)

np.random.seed(123)
result_2 = np.random.randint(1, 50, size=5)

print(result_1)
print(result_2)
print(np.array_equal(result_1, result_2))
```

The final comparison returns **True** because both sequences were generated after resetting the generator to the same seed.

10. Why Reproducibility Is Important

Reproducibility means that an experiment can be run again and produce the same expected random sequence when the same setup is used. This helps with debugging, testing, teaching, comparison of experiments, and verification of results.

11. Sampling with Probabilities

The `p` argument of `np.random.choice()` allows different categories to have different selection probabilities.

```
categories = np.array(['Low','Medium','High'])
sample = np.random.choice(
    categories, size=10, p=[0.5, 0.3, 0.2]
)
print(sample)
```

In this example, the requested probabilities are 50% for Low, 30% for Medium, and 20% for High. The probabilities must form a valid probability distribution and sum to 1.

12. Random Sampling: With vs Without Replacement

Method	Meaning	Typical Use
replace=True	An item may be selected again.	Sampling where repeated selections are allowed
replace=False	An item cannot be selected again in the same sample.	Selecting unique records/items

13. Practical Data Science Applications

NumPy random sampling can support:
• Synthetic dataset creation
• Simulation and experimentation
• Random sampling of records
• Testing algorithms with generated data
• Reproducible experiments
• Demonstrations and learning exercises

14. Common Mistakes

1. Forgetting that `randint()` excludes its upper boundary.
2. Assuming pseudo-random values are truly unpredictable in every security context.
3. Forgetting to set a seed when reproducible results are required.
4. Using an invalid probability list in `choice()`.

5. Requesting more unique samples than the population when `replace=False`.

15. Step-by-Step Problem-Solving Workflow

Step 1: Decide what type of random data is needed.
Step 2: Select the appropriate NumPy random function.
Step 3: Define the range, sample size, or probability distribution.
Step 4: Set a seed if reproducibility is required.
Step 5: Generate the data or sample.
Step 6: Inspect the output and verify its shape and values.
Step 7: Use the generated data in the required analysis.

16. Complete Practical Program

```
import numpy as np

np.random.seed(42)

# Random integers
random_integers = np.random.randint(1, 101, size=10)
print("Random integers:", random_integers)

# Random floats
random_floats = np.random.random(5)
print("Random floats:", random_floats)

# Random student sample
students = np.array(["Aman", "Priya", "Rahul", "Neha", "Karan"])
sample = np.random.choice(students, size=3, replace=False)
print("Student sample:", sample)

# Simulated marks
np.random.seed(100)
marks = np.random.randint(40, 101, size=10)
print("Marks:", marks)
print("Mean:", marks.mean())
print("Minimum:", marks.min())
print("Maximum:", marks.max())

# Reproducibility
np.random.seed(123)
a = np.random.randint(1, 50, size=5)
np.random.seed(123)
b = np.random.randint(1, 50, size=5)
print("Same results:", np.array_equal(a, b))
```

17. Expected Output Interpretation

The exact random values depend on the selected seed and sequence of operations. With a fixed seed, the same code sequence can reproduce the same pseudo-random values. The student-mark example can then be used for basic numerical summaries such as mean, minimum, and maximum.

18. Interview Questions and Answers

Q1. Why are random numbers useful in data science?
They are useful for simulation, sampling, testing, experimentation, and synthetic data generation.

Q2. What is a random seed?
A seed initializes a pseudo-random generator so that a repeatable sequence can be produced.

Q3. Why is reproducibility important?
It allows experiments to be repeated and results to be checked consistently.

Q4. Which NumPy function selects random values from an array?
`np.random.choice()`.

Q5. What is the difference between randint() and random()?
`randint()` generates integers in a selected range, while `random()` generates floating-point values from 0 up to 1.

Q6. What does replace=False mean?
It prevents the same array element from being selected more than once in that sampling operation.

19. Quick Revision Table

Function	Purpose
np.random.randint()	Generate random integers
np.random.random()	Generate random floats from 0 to 1
np.random.uniform()	Generate random floats in a specified range
np.random.choice()	Select random elements from an array
np.random.seed()	Make pseudo-random sequences reproducible

20. Top 5 Skills

1. **NumPy**
2. **Random Data Generation**
3. **Random Sampling**
4. **Reproducible Data Analysis**
5. **Python Data Science**

21. Feedback

The task was very helpful for understanding how **NumPy random functions** can be used in data science experiments. I learned how to generate random integers and floating-point numbers, select random samples from arrays, create simulated student data, and use random seeds for reproducible results. The practical examples improved my understanding of **simulation, sampling, and reproducible data analysis**.